

# Redes Neuronales

Del aprendizaje supervisado al perceptrón

Sergio Hernández, PhD  
shernandez@ucm.cl



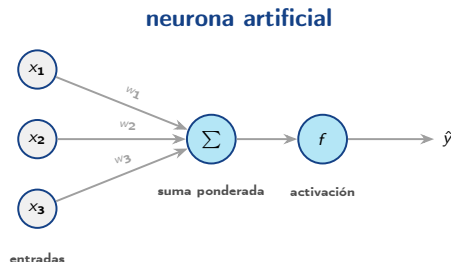
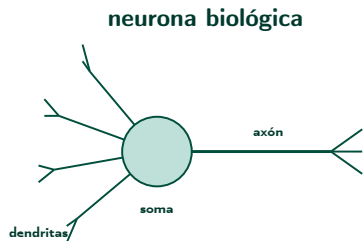
Magíster en Data Science — Universidad Católica del Maule

1. De la neurona biológica a la artificial
2. Aprendizaje supervisado: regresión
3. Capacidad, sub-ajuste y sobreajuste
4. El perceptrón y su regla de aprendizaje
5. De la decisión dura a la probabilidad
6. El límite: el problema XOR

## Objetivo de la clase

Construir la **primera neurona artificial** completa: su forma funcional, cómo se ajustan sus parámetros a partir de datos y — lo más importante — **qué no puede hacer**.

# De la neurona biológica a la artificial



dendritas  $\rightarrow$  entradas   soma  $\rightarrow$  suma ponderada   umbral de disparo  $\rightarrow$  activación

**Idea clave:** La analogía motiva el modelo, pero no lo justifica. El cerebro tiene  $\sim 10^{11}$  neuronas con dinámica temporal, neurotransmisores y plasticidad local: nada de eso está en  $f(w^T x)$ . Es un **modelo inspirado** en la biología, no una simulación de ella.

# ¿Qué significa aprender?

## Agente que aprende

Un agente **aprende** si, dado un modelo y un criterio de optimización, mejora su desempeño al recibir nueva información.

# ¿Qué significa aprender?

## Agente que aprende

Un agente **aprende** si, dado un modelo y un criterio de optimización, mejora su desempeño al recibir nueva información.

- ▶ Del conjunto de **entrenamiento** el agente construye un mapa entre las características observadas y la respuesta.
- ▶ Ante datos nuevos, cuya respuesta desconoce, usa ese mapa para **generalizar**.

$$\min_w \frac{1}{n} \sum_{i=1}^n \ell(f_w(x_i), y_i)$$

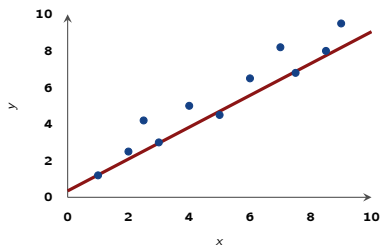
Todo lo que sigue en esta clase es *una* elección concreta de  $f_w$  y de  $\ell$ .

**Idea clave:** Ajustar bien el conjunto de entrenamiento es fácil y no es el objetivo. El objetivo es el error en datos **no vistos**.

# Aprendizaje supervisado: dos tareas

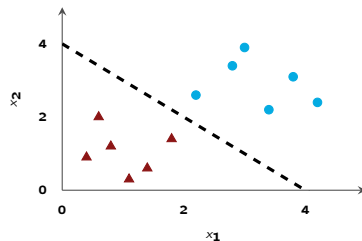
Dado  $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ , la diferencia está en la **naturaleza de  $y$** :

**Regresión:**  $y \in \mathbb{R}$



predecir un **valor**

**Clasificación:**  $y \in \{c_1, \dots, c_k\}$



asignar una **categoría**

**Idea clave:** Misma maquinaria, distinto  $\ell$ : el error cuadrático para valores continuos, la entropía cruzada para categorías. Lo veremos al final de la clase.

- ▶ Para regresión no lineal en una variable, ajustamos un polinomio de orden  $p$ :

$$y = f(x) + \epsilon = w_0 + \sum_{j=1}^p w_j x^j + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

- ▶ Para regresión no lineal en una variable, ajustamos un polinomio de orden  $p$ :

$$y = f(x) + \epsilon = w_0 + \sum_{j=1}^p w_j x^j + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

- ▶ Los parámetros  $w = [w_0, \dots, w_p]$  se obtienen por **mínimos cuadrados**:

$$E_p(w) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2$$



- ▶ Para regresión no lineal en una variable, ajustamos un polinomio de orden  $p$ :

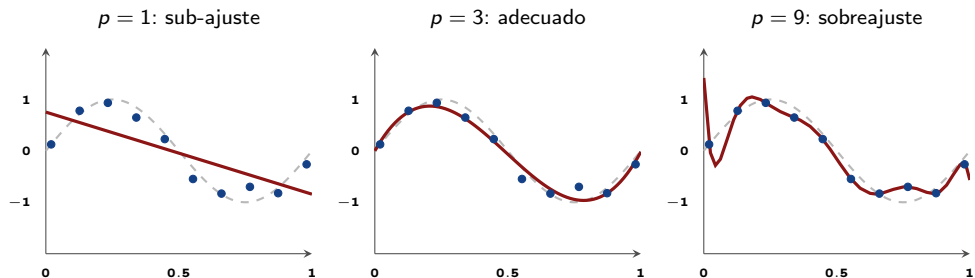
$$y = f(x) + \epsilon = w_0 + \sum_{j=1}^p w_j x^j + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

- ▶ Los parámetros  $w = [w_0, \dots, w_p]$  se obtienen por **mínimos cuadrados**:

$$E_p(w) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2$$

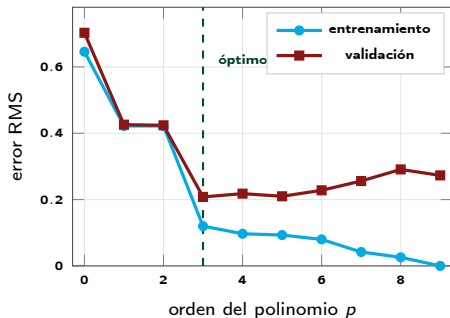
- ▶ El modelo es **no lineal en  $x$**  pero **lineal en  $w$** : por eso tiene solución cerrada,  $w^* = (X^T X)^{-1} X^T y$ .
- ▶  $p$  no se aprende de los datos: es un **hiperparámetro** que fija la *capacidad* del modelo.

# Los mismos datos, tres capacidades



**Idea clave:** Con  $p = 9$  y 10 puntos el ajuste es **perfecto** sobre los datos de entrenamiento (error cero) y sin embargo el modelo es peor. El error de entrenamiento no mide lo que nos importa.

# La curva en U: capacidad vs. generalización



- ▶ El error de **entrenamiento** baja de forma monótona: más capacidad siempre ajusta mejor lo ya visto.
- ▶ El error de **validación** baja, toca un mínimo y vuelve a subir: ahí empieza el sobreajuste.
- ▶ La distancia entre ambas curvas es la **brecha de generalización**.

Herramientas para controlarla: más datos, regularización, detención temprana. Es el tema de la clase 1.4.

**Idea clave:** La capacidad óptima no es la máxima ni la mínima: se **elige**, y se elige midiendo sobre datos que el modelo no usó para ajustarse.

- ▶ Clasificación binaria con entradas  $x \in \mathbb{R}^D$  y etiqueta  $y \in \{-1, +1\}$ .
- ▶ Los  $w_j$  son los **pesos** y  $w_0$  el **sesgo**.

$$f(x) = f\left(w_0 + \sum_{j=1}^D w_j x_j\right) = f(w^T x)$$

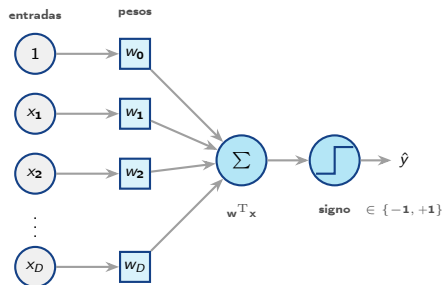
# El perceptrón

- ▶ Clasificación binaria con entradas  $x \in \mathbb{R}^D$  y etiqueta  $y \in \{-1, +1\}$ .
- ▶ Los  $w_j$  son los **pesos** y  $w_0$  el **sesgo**.

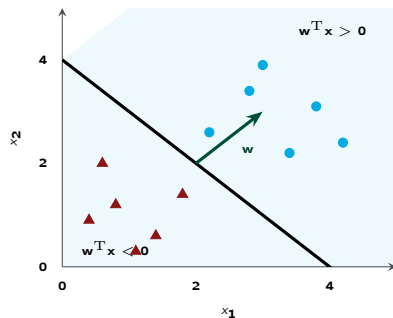
$$f(x) = f\left(w_0 + \sum_{j=1}^D w_j x_j\right) = f(w^T x)$$

- ▶ Rosenblatt (1958) usa como activación la **función signo**:

$$f(x) = \begin{cases} +1 & w^T x \geq 0 \\ -1 & w^T x < 0 \end{cases}$$



# Qué significa geométricamente



- ▶ La ecuación  $w^T x = 0$  define un **hiperplano**: una recta en  $\mathbb{R}^2$ , un plano en  $\mathbb{R}^3$ .
- ▶ El vector  $w$  es **perpendicular** a ese hiperplano y apunta hacia la clase +1.
- ▶ El sesgo  $w_0$  desplaza el hiperplano sin cambiar su orientación.
- ▶ Clasificar es preguntar **de qué lado** cae el punto.

**Idea clave:** El perceptrón solo puede trazar **una** frontera recta. Toda la clase se juega en esa limitación.

- ▶ Los patrones bien clasificados aportan error cero. Para los mal clasificados ( $i \in \mathcal{M}$ ) se cumple  $w^T x_i y_i < 0$ , así que minimizamos

$$E(w) = - \sum_{i \in \mathcal{M}} w^T x_i y_i \geq 0$$

- ▶ Los patrones bien clasificados aportan error cero. Para los mal clasificados ( $i \in \mathcal{M}$ ) se cumple  $w^T x_i y_i < 0$ , así que minimizamos

$$E(w) = - \sum_{i \in \mathcal{M}} w^T x_i y_i \geq 0$$

- ▶ El entrenamiento usa **descenso de gradiente**:

$$w^{\tau+1} = w^{\tau} - \eta \nabla E(w^{\tau}) = w^{\tau} + \eta \sum_{i \in \mathcal{M}} x_i y_i$$



# La regla de aprendizaje

- ▶ Los patrones bien clasificados aportan error cero. Para los mal clasificados ( $i \in \mathcal{M}$ ) se cumple  $w^T x_i y_i < 0$ , así que minimizamos

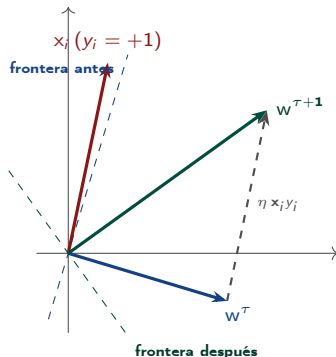
$$E(w) = - \sum_{i \in \mathcal{M}} w^T x_i y_i \geq 0$$

- ▶ El entrenamiento usa **descenso de gradiente**:

$$w^{\tau+1} = w^{\tau} - \eta \nabla E(w^{\tau}) = w^{\tau} + \eta \sum_{i \in \mathcal{M}} x_i y_i$$

- ▶  $\eta > 0$  es la **tasa de aprendizaje** y  $\tau$  el número de iteración.
- ▶ Solo los ejemplos **mal clasificados** modifican los pesos: un patrón correcto no aporta nada al gradiente.

# Por qué funciona la regla



- ▶  $x_i$  pertenece a la clase  $+1$  pero cae del lado negativo de  $w^\tau$ : está **mal clasificado**.
- ▶ La actualización suma  $\eta x_i y_i$ , es decir **rota  $w$  hacia el patrón**.
- ▶ Tras el paso,  $w^{\tau+1 T} x_i = w^\tau T x_i + \eta \|x_i\|^2$ : el producto *siempre* aumenta.
- ▶ Si la etiqueta fuera  $-1$ , la regla restaría  $x_i$  y alejaría el vector.

**Idea clave:** Cada corrección mejora el margen *de ese patrón*, aunque pueda estropear otros. La garantía global la da el teorema de convergencia.

## Teorema de convergencia

Si las clases son **linealmente separables**, la regla del perceptrón encuentra una solución en un número **finito** de pasos, cualquiera sea la inicialización.

## La letra chica

- ▶ Si *no* son separables, el algoritmo **no converge**: oscila indefinidamente.
- ▶ La solución encontrada no es única ni la de mayor margen: depende del orden de los datos.
- ▶ No se extiende de forma natural a más de dos clases.
- ▶ La salida es una decisión dura: no dice *cuán seguro* está.

Los dos últimos problemas se resuelven cambiando la función de activación. El primero, cambiando la arquitectura.

- ▶ El signo devuelve un lado, no una confianza. Modelemos ahora  $p(y = 1 \mid x)$ .
- ▶ Cambiamos de codificación: de aquí en adelante  $y \in \{0, 1\}$  en lugar de  $\{-1, +1\}$ .
- ▶ Postulamos que el **logaritmo de la razón de momios** es lineal en  $x$ :

$$\log\left(\frac{p(y = 1 \mid x)}{p(y = 0 \mid x)}\right) = w^T x$$

- ▶ El signo devuelve un lado, no una confianza. Modelemos ahora  $p(y = 1 \mid x)$ .
- ▶ Cambiamos de codificación: de aquí en adelante  $y \in \{0, 1\}$  en lugar de  $\{-1, +1\}$ .
- ▶ Postulamos que el **logaritmo de la razón de momios** es lineal en  $x$ :

$$\log\left(\frac{p(y = 1 \mid x)}{p(y = 0 \mid x)}\right) = w^T x$$

- ▶ Despejando se obtiene la **función logística** (sigmoide):

$$p(y = 1 \mid x) = \sigma(w^T x) = \frac{\exp(w^T x)}{1 + \exp(w^T x)} = \frac{1}{1 + \exp(-w^T x)}$$
$$p(y = 0 \mid x) = 1 - \sigma(w^T x)$$

Las dos expresiones de  $\sigma$  son equivalentes: basta dividir numerador y denominador por  $\exp(w^T x)$ . Ojo con el **signo del exponente**.

# La función sigmoide

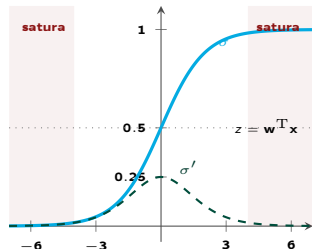
- ▶  $\sigma : \mathbb{R} \mapsto (0, 1)$ , así que la salida se lee como probabilidad.
- ▶ Es continua y diferenciable: habilita el descenso de gradiente, que el escalón del perceptrón impedía.
- ▶ Su derivada se escribe en términos de sí misma:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\frac{\partial}{\partial w} \sigma(w^T x) = \sigma(1 - \sigma) x$$

El factor  $x$  viene de la regla de la cadena: no lo olvide al derivar respecto de los pesos.

**Idea clave:** En las colas  $\sigma' \rightarrow 0$ : el gradiente se desvanece y el aprendizaje se detiene. Este defecto reaparecerá al apilar capas y motivará ReLU.



## La pérdida: entropía cruzada

La verosimilitud de una etiqueta Bernoulli es  $p^y(1-p)^{1-y}$ . Tomando  $-\log$  y promediando:

$$E(w) = \frac{1}{n} \sum_{i=1}^n \left[ -y_i \log \sigma(w^T x_i) - (1 - y_i) \log (1 - \sigma(w^T x_i)) \right]$$

# La pérdida: entropía cruzada

La verosimilitud de una etiqueta Bernoulli es  $p^y(1-p)^{1-y}$ . Tomando  $-\log$  y promediando:

$$E(w) = \frac{1}{n} \sum_{i=1}^n \left[ -y_i \log \sigma(w^T x_i) - (1 - y_i) \log (1 - \sigma(w^T x_i)) \right]$$

## Regresión (error cuadrático)

$$\nabla E = \frac{2}{n} \sum_i (\hat{y}_i - y_i) x_i$$

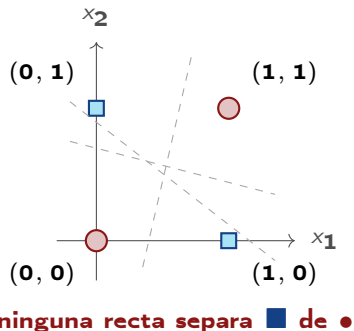
## Clasificación (entropía cruzada)

$$\nabla E = \frac{1}{n} \sum_i (\sigma(w^T x_i) - y_i) x_i$$

**Idea clave:** Distinto modelo y distinta pérdida, **misma forma del gradiente:** (predicción – objetivo)  $\times$  entrada. No es casualidad, y es lo que hace que un mismo optimizador sirva para todo el curso.



# El problema XOR



| $x_1$ | $x_2$ | $y = x_1 \oplus x_2$ |
|-------|-------|----------------------|
| 0     | 0     | 0                    |
| 0     | 1     | 1                    |
| 1     | 0     | 1                    |
| 1     | 1     | 0                    |

- ▶ AND y OR *sí* son linealmente separables; XOR no.
- ▶ Un perceptrón no puede aprender la función lógica más simple que no es lineal.

**Idea clave:** Bastan **dos** rectas para resolverlo: una capa oculta de dos neuronas más una de salida. Ese es exactamente el perceptrón multicapa de la próxima clase.

- ▶ En *Perceptrons* (1969), Minsky y Papert demostraron formalmente esta limitación.
- ▶ El libro también señaló que no existía entonces un algoritmo de entrenamiento para redes **multicapa**.
- ▶ El financiamiento a redes neuronales se desplomó: es el **primer invierno de la IA** que vimos en la clase 1.1.

## Lo que faltaba

La respuesta llegó en 1986 con la popularización de la **retropropagación** (Rumelhart, Hinton y Williams): una forma eficiente de calcular el gradiente en redes de varias capas.

Entre el diagnóstico del problema y su solución pasaron **diecisiete años**.

**Idea clave:** La limitación era real, pero se leyó como una condena del enfoque completo en vez de como un problema de arquitectura. Conviene distinguir entre *este modelo no puede* y *esta idea no sirve*.

| Concepto             | Idea clave                                                           |
|----------------------|----------------------------------------------------------------------|
| Neurona artificial   | $f(w^T x)$ : inspirada en la biología, no una simulación             |
| Aprendizaje superv.  | Regresión (y continua) y clasificación (y categórica)                |
| Capacidad            | El orden $p$ es un hiperparámetro, no se ajusta con los datos        |
| Sobreajuste          | Error de entrenamiento $\downarrow$ , error de validación $\uparrow$ |
| Perceptrón           | Hiperplano $w^T x = 0$ ; $w$ es su normal                            |
| Regla de aprendizaje | Solo los errores actualizan: $w \leftarrow w + \eta x_i y_i$         |
| Sigmoide             | Decisión dura $\rightarrow$ probabilidad; derivable, pero satura     |
| Gradiente            | (predicción – objetivo) $\times$ entrada, en ambos casos             |
| XOR                  | El límite estructural: una sola frontera lineal                      |

- Próxima clase: apilar neuronas en capas y entrenar la red completa con retropropagación.

- ▶ Bishop, *Pattern Recognition and Machine Learning*, cap. 1 (regresión polinomial) y 4 (modelos lineales de clasificación).
- ▶ Rosenblatt (1958), *The perceptron: a probabilistic model for information storage and organization in the brain*, *Psychological Review* 65(6).
- ▶ Minsky & Papert (1969), *Perceptrons*, MIT Press.
- ▶ Rumelhart, Hinton & Williams (1986), *Learning representations by back-propagating errors*, *Nature* 323.
- ▶ Prince, *Understanding Deep Learning*, cap. 2–3  
<https://udlbook.github.io/udlbook/>
- ▶ Zhang, Lipton, Li & Smola, *Dive into Deep Learning*, cap. 3–4  
<https://d2l.ai/>

¿Preguntas?